

This document describes only the React front end and WebSocket communication used in the rover control system. It explains how the dashboard is structured, how the user interacts with it, how telemetry is displayed, and how commands and data are exchanged through WebSockets.

1. System Overview

The front end is a React application built with Vite. Its main purpose is to provide the operator with a simple mission-control dashboard. The interface allows the operator to select the rover operating mode, send movement commands, select motor speed, view the current motion status, monitor telemetry, and see the connection state.

The browser communicates with the control backend through a WebSocket connection. WebSockets are used because the rover dashboard needs two-way communication: the front end sends commands to the backend, while the backend continuously sends telemetry and warning messages back to the front end.

2. Main Technologies

- React: Builds the user interface from reusable components.
- Vite: Provides the development server and build system for the React project.
- Motion: Provides simple interface animations such as fade-in effects, button feedback, and status indicators.
- Browser WebSocket API: Maintains the live connection between the React dashboard and the backend.
- JSON: Provides the message format used for WebSocket commands and telemetry.

3. Front-End File Structure

```
react/
├── src/
│   ├── App.jsx
│   ├── main.jsx
│   ├── index.css
│   ├── App.css
│   ├── components/
│   │   ├── Header.jsx
│   │   ├── CameraFeed.jsx
│   │   ├── ControlPanel.jsx
│   │   └── SensorPanel.jsx
│   └── hooks/
│       └── useCarSocket.js
├── package.json
└── vite.config.js
```

The important files for this documentation are App.jsx, the four UI components, and useCarSocket.js. The WebSocket server side is implemented in backend_server.py, but only its WebSocket responsibilities are covered here.

4. React Front-End Structure

The application starts in main.jsx. React renders the App component inside the HTML element with the id root. App.jsx then acts as the main container and connects the individual dashboard components together.

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
)
```

4.1 App.jsx

App.jsx keeps the main application state and passes the required information to each component. The operating mode starts as manual. The useCarSocket hook supplies the WebSocket connection state, telemetry, warning state, and functions for sending drive and mode commands.

```
const [mode, setMode] = useState("manual");  
const { connected, telemetry, warning, sendDrive, sendMode }  
  = useCarSocket();
```

When the operator changes the mode, handleSetMode updates the React state and also sends the new mode through the WebSocket. This keeps the visual state of the dashboard synchronized with the command sent to the control system.

4.2 Header Component

Header.jsx displays the application title, mission-control subtitle, and connection state. The connected property determines whether the interface displays CONNECTED or DISCONNECTED. A warning banner is also displayed when a warning message is received.

- CONNECTED means the browser currently has an open WebSocket connection.
- DISCONNECTED means the WebSocket is closed or could not be established.
- Warnings are displayed when the WebSocket receives a warning message.

4.3 Camera Feed

CameraFeed.jsx provides the visual camera section of the dashboard. The component displays an image whose source is the local video endpoint at <http://localhost:5000/video>. The camera connection itself is not implemented with the WebSocket hook; therefore, it is treated separately from the command and telemetry WebSocket.

4.4 Control Panel

ControlPanel.jsx contains the operator controls. It provides manual/autonomous mode selection, directional movement buttons, a stop button, motor-speed selection, and a motion-status indicator.

5. Control Panel and User Interaction

5.1 Operating Modes

The control panel supports two modes: MANUAL and AUTONOMOUS. The selected mode is stored in the mode state in App.jsx. Selecting a mode calls `setMode`, which eventually calls `sendMode` from the WebSocket hook.

```
const handleSetMode = (newMode) => {  
  setMode(newMode);  
  sendMode(newMode);  
};
```

When autonomous mode is selected, the directional buttons are disabled. This prevents the operator from manually sending forward, backward, left, or right commands while autonomous control is active. The STOP button remains available.

5.2 Movement Commands

The movement buttons call `handleCommand()`. The command is used to update the local motion status and is then passed to `sendDrive()`. The available movement commands are FORWARD, BACKWARD, LEFT, RIGHT, and STOP.

```
sendDrive(command, SPEED_VALUES[speed]);
```

The speed selector converts the labels LOW, MEDIUM, and HIGH into numeric values. The current implementation uses 3, 6, and 9 respectively. The selected value is included with the drive WebSocket message.

5.3 Motion Status

The dashboard keeps a simple local `motionStatus` value. When STOP is selected, the status becomes STOPPED. For other commands, the displayed status becomes the command that was selected. This provides immediate visual feedback to the operator.

6. Sensor Panel

SensorPanel.jsx displays real-time telemetry received from the WebSocket connection. It reads the telemetry object supplied by App.jsx and shows battery voltage, current, accelerometer values, and gyroscope values.

```
const voltage = telemetry?.voltage ?? "--";  
const current = telemetry?.current ?? "--";  
const accel = telemetry?.accel ?? { x: 0, y: 0, z: 0 };  
const gyro = telemetry?.gyro ?? { x: 0, y: 0, z: 0 };
```

- Voltage is displayed in volts (V).
- Current is displayed in amperes (A).
- Accelerometer data contains X, Y, and Z values.
- Gyroscope data contains X, Y, and Z values.

If telemetry has not arrived yet, the component uses fallback values. This prevents the interface from crashing when the connection has not supplied sensor data.

7. WebSocket Communication

The main WebSocket implementation on the React side is useCarSocket.js. It is implemented as a custom React hook so that connection management and message processing are kept separate from the visual components.

7.1 Connection

The default WebSocket address is ws://localhost:8765. When the hook is created, it opens a browser WebSocket connection using the URL.

```
const WS_URL = "ws://localhost:8765";

const ws = new WebSocket(url);
```

The hook tracks the connection with the connected state. The onopen callback sets connected to true, while onclose sets it to false.

7.2 Automatic Reconnection

If the connection closes, the hook automatically attempts to reconnect after two seconds. This is useful for a rover system because the communication link may temporarily disappear. The operator does not need to refresh the entire dashboard after every temporary connection failure.

```
ws.onclose = () => {
  setConnected(false);
  reconnectTimer.current = setTimeout(connect, 2000);
};
```

The cleanup function clears the reconnect and warning timers and closes the WebSocket when the React component is removed.

7.3 Sending Commands

The send function checks that the WebSocket is open before sending data. Messages are converted to JSON strings before being transmitted.

```
const send = useCallback((payload) => {
  if (wsRef.current?.readyState === WebSocket.OPEN) {
    wsRef.current.send(JSON.stringify(payload));
  }
}, []);
```

Drive commands use the following logical structure:

```
{
  "type": "drive",
  "direction": "forward",
  "speed": 6
}
```

The direction is converted to lowercase before transmission. Mode commands use a similar structure:

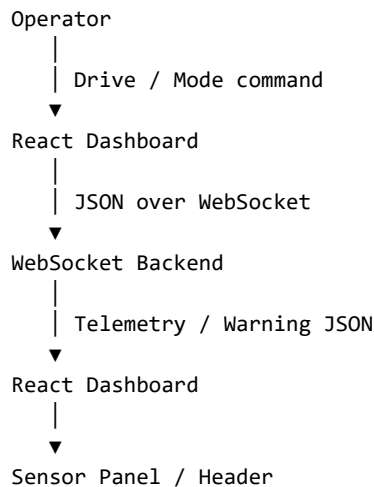
```
{
  "type": "mode",
  "mode": "manual"
}
```

7.4 Receiving Telemetry

The WebSocket onmessage callback receives messages from the backend. The message is parsed as JSON. If the type is telemetry, the hook updates the telemetry React state. This automatically causes SensorPanel to re-render with the newest values.

8. WebSocket Message Flow

Communication is bidirectional. Commands travel from the browser to the WebSocket backend, while telemetry and warnings travel from the backend to the browser.



8.1 Telemetry Message Format

The backend sends telemetry using a JSON object with type set to telemetry. The accelerometer and gyroscope values are represented as objects with x, y, and z fields.

```
{
  "type": "telemetry",
  "voltage": 12.40,
  "current": 1.23,
  "accel": {
    "x": 0.10,
    "y": 0.20,
    "z": 9.81
  },
  "gyro": {
    "x": 0.10,
    "y": 0.20,
    "z": 0.30
  }
}
```

8.2 Warning Message Format

Warnings use a separate message type. The current backend can send LOW_VOLTAGE or OVERCURRENT. The React hook stores the warning and Header displays a readable warning banner.

```
{
  "type": "warning",
```

```
"code": "LOW_VOLTAGE"  
}
```

The warning is automatically cleared after five seconds. This prevents an old warning from remaining permanently visible after the condition is no longer active.

9. Backend WebSocket Responsibilities

The WebSocket portion of `backend_server.py` is responsible for accepting browser connections, maintaining a set of connected clients, receiving JSON commands, and broadcasting telemetry or warning messages. The server listens on port 8765.

- Register a client when a WebSocket connection is opened.
- Remove the client when the connection closes.
- Read and parse incoming JSON messages.
- Process drive and mode message types.
- Broadcast telemetry to connected clients.
- Broadcast warning messages when required.